Universidad del Valle de Guatemala

Facultad de Ingeniería · Departamento de Ciencias de la Computación

CC3084 – Data Science · Semestre II, 2026

Identificación de especies de mosquitos mediante detección y clasificación de imágenes

Proyecto 2 – Informe de resultados

Reto MosquitoAlert Challenge 2023 (AIcrowd)

Guatemala, octubre de 2026

Índice

Índice 2
1. Introducción 4
1.1 Situación problemática 4
1.2 Problema científico 4
2. Objetivos 5
2.1 Objetivo general 5
2.2 Objetivos específicos 5
3. Marco teórico 5
3.1 Procesamiento de imágenes digitales 5
3.2 Perspectiva de la especialidad: salud pública y entomología 6
3.3 Algoritmos de aprendizaje automático 7
3.3.1 Redes neuronales convolucionales y ajuste fino 7
3.3.2 Transformers de visión y DINOv2 7
3.3.3 Clasificadores sobre características 7
3.3.4 Detección de objetos con YOLOv8 7
3.3.5 Manejo del desbalance de clases 8
3.3.6 Métricas de evaluación 8
3.4 Antecedentes: estudios previos 8
4. Metodología 9
4.1 Pasos seguidos por el grupo 9
4.2 Selección de los conjuntos de entrenamiento, validación y prueba 10
4.3 Selección de los algoritmos y razones 10
4.4 Herramientas utilizadas 10
Recursos de cómputo 10
Lenguajes, bibliotecas y paquetes 10
4.5 Del plan preliminar a la implementación 11
5. Resultados y análisis de resultados 11
5.1 Características del conjunto de datos original 11
5.2 Limpieza y preprocesamiento 12
5.3 Análisis exploratorio 12
5.3.1 Variables cuantitativas 12
5.3.2 Tamaño relativo del mosquito por clase 13
5.3.3 Distribución de clases 15

5.3.4 Cruce de variables 16
5.3.5 Valores atípicos y faltantes 17
5.3.6 Inspección visual de las imágenes 18
5.3.7 Hallazgos del análisis exploratorio y decisiones derivadas 19
5.4 Ajuste de parámetros de cada algoritmo 19
5.4.1 Medición de rendimiento de la GPU 19
5.4.2 Clasificadores sobre DINOv2 19
5.4.3 CNN con ajuste fino 20
5.4.4 Detector YOLOv8n 20
5.5 Comparación de los algoritmos 21
5.5.1 Desempeño por clase 22
5.5.2 Análisis de errores 23
5.5.3 Costo computacional frente a desempeño 24
5.5.4 Combinación y acuerdo entre modelos 24
5.6 Resultados del detector 24
5.7 Aplicación desarrollada 25
6. Conclusiones 26
6.1 Discusión sobre la efectividad de los algoritmos 26
6.2 Limitaciones y trabajo futuro 27
Referencias 27

1. Introducción

Los mosquitos figuran entre los animales que más muertes humanas causan de forma indirecta (Organización Mundial de la Salud [OMS], 2024), pues varias de sus especies transmiten enfermedades como el dengue, el zika, la chikungunya, la fiebre amarilla, el virus del Nilo Occidental y la malaria. Conocer qué especies están presentes en un territorio y cómo se distribuyen es la base de la vigilancia entomológica y del control de vectores. Tradicionalmente esa vigilancia depende de trampas y de expertos que identifican los ejemplares uno a uno, lo cual es lento y costoso.

La plataforma de ciencia ciudadana MosquitoAlert recibe fotografías de mosquitos tomadas por personas comunes con sus teléfonos. Estas imágenes tienen fondos, iluminación, enfoque y tamaños muy variados, lo que hace difícil su revisión manual a gran escala. El reto MosquitoAlert Challenge 2023, publicado en AIcrowd, propone automatizar el proceso: localizar el mosquito en la fotografía y clasificarlo en una de seis categorías, con cajas delimitadoras anotadas por entomólogos.

Este informe documenta el proyecto desarrollado en el curso CC3084 (Data Science) para abordar ese reto. Se construyó un sistema de dos etapas: un detector YOLOv8n de una sola clase que localiza el mosquito y un clasificador que trabaja sobre el recorte detectado. Se compararon seis clasificadores (dos familias de aprendizaje profundo y cuatro variantes sobre características de DINOv2) y se desarrolló una aplicación local en Streamlit para probarlos.

El documento sigue la estructura solicitada: objetivos, marco teórico, metodología, resultados y análisis de resultados (datos originales, limpieza y preprocesamiento, análisis exploratorio, ajuste de parámetros, comparación de algoritmos y descripción de la aplicación) y conclusiones.

1.1 Situación problemática

La identificación tradicional de los vectores de enfermedades depende de la inspección manual de expertos, un proceso lento y costoso que dificulta a las instituciones de salud pública el monitoreo, la detección y la reacción rápida ante brotes de enfermedades transmitidas por mosquitos, como el zika, el dengue o la chikungunya.

Plataformas como Mosquito Alert buscan reducir estas limitaciones permitiendo que la ciudadanía participe y genere una gran cantidad de fotografías, para las cuales la validación humana experta no puede mantener el mismo ritmo. Revisar cada imagen a mano no es sostenible a largo plazo (Pataki et al., 2021), y la automatización mediante modelos de visión artificial es una herramienta clave para apoyar la toma de decisiones a tiempo.

1.2 Problema científico

¿Es posible desarrollar un modelo de visión artificial capaz de detectar y clasificar correctamente la especie o género de un mosquito, entre las seis clases del reto, a partir de fotografías tomadas por ciudadanos no expertos, con condiciones de iluminación, ángulo y calidad muy variables? El problema se concentra en tres retos técnicos que emergen directamente de los datos:

- **Objetos pequeños dentro de la imagen:** el mosquito ocupa solo una fracción reducida de la fotografía (la mediana de la proporción del área de la caja es 7.3 % del área de la imagen), lo que dificulta la detección y la extracción de rasgos morfológicos.
- **Fuerte desbalance de clases:** aegypti (47 imágenes) y anopheles (84) están drásticamente subrepresentadas frente a albopictus (4,611) y culex (4,561), lo que sesga a cualquier modelo hacia las clases mayoritarias si no se corrige.
- **Variabilidad no controlada de captura:** celulares, fondos, iluminación y ángulos distintos obligan al modelo a generalizar más allá de condiciones de laboratorio.

El problema consiste, por tanto, en determinar en qué medida la detección de objetos (YOLOv8) combinada con clasificación por transferencia de aprendizaje puede alcanzar un desempeño aceptable, medido con F1 macro (la métrica oficial del reto), pese a estas tres condiciones, y qué estrategias de balanceo resultan más efectivas para mitigar el sesgo hacia las clases mayoritarias.

2. Objetivos

2.1 Objetivo general

Desarrollar un modelo para la localización y clasificación automatizada de seis especies y géneros de mosquitos vectores a partir de fotografías de ciencia ciudadana, evaluando estrategias para mitigar el desbalance de clases y la variabilidad visual.

2.2 Objetivos específicos

- Analizar el desbalance de las clases en el conjunto de datos para entender qué especies tienen pocas imágenes.
- Evaluar la calidad y la variación de las imágenes (tamaños, iluminación) para definir técnicas de aumento de datos.
- Construir y evaluar un modelo base para medir el desempeño inicial en la clasificación de las seis especies.
- Comparar distintos algoritmos (características de DINOv2 con clasificadores planos y jerárquicos, y CNN con ajuste fino) según F1 macro, exactitud balanceada, exactitud, desempeño por clase y costo de cómputo.
- Construir una aplicación local que integre el detector y los clasificadores para probar los modelos con imágenes propias.

3. Marco teórico

3.1 Procesamiento de imágenes digitales

Una imagen digital es una matriz de píxeles; en color, cada píxel tiene tres canales (rojo, verde y azul) con valores entre 0 y 255. Los algoritmos de visión por computadora transforman esa matriz en información útil, como la ubicación de un objeto o su categoría. Antes de entrenar un modelo suele aplicarse un

preprocesamiento: recorte de la región de interés, redimensionamiento a un tamaño fijo, normalización de los valores de los canales y decodificación eficiente de los archivos.

El aumento de datos (data augmentation) genera variantes de las imágenes de entrenamiento mediante transformaciones como recortes aleatorios, volteos, rotaciones y cambios de brillo, contraste o color. Su objetivo es que el modelo sea robusto a la variación natural y no memorice las imágenes, y es especialmente útil cuando hay pocas muestras de una clase.

Se distinguen dos tareas relacionadas. La clasificación asigna una etiqueta a la imagen completa. La detección de objetos, además, estima una caja delimitadora que indica dónde está el objeto. La calidad de una caja se mide con la intersección sobre la unión (IoU): el área de solapamiento entre la caja predicha y la real dividida por el área de su unión. El reto exige IoU ≥ 0.75 para considerar correcta una localización.

3.2 Perspectiva de la especialidad: salud pública y entomología

Distinguir con precisión la especie o género de un mosquito es importante para la salud pública porque no todos transmiten las mismas enfermedades ni responden igual a los métodos de control. Solo las hembras necesitan alimentarse de sangre para el desarrollo de sus huevos, por lo que son las principales responsables de la transmisión; identificar correctamente la especie y el sexo es esencial para evaluar el riesgo y orientar la vigilancia y el control de vectores (Duan et al., 2026; OMS, 2024).

La siguiente tabla resume las enfermedades asociadas a cada clase y los rasgos visuales con que un experto las distingue, según las fuentes consultadas por el grupo (Paixão et al., 2018; Nebbak et al., 2022; Das, 2013; Richter-Boix, 2021; Mosquito Alert, s. f.).

Tabla 1. *Clases del reto: enfermedades asociadas y rasgos distintivos.*

Clase	Enfermedades asociadas	Rasgos visuales distintivos
Aedes aegypti	Chikungunya, dengue, filariasis linfática, fiebre del Valle del Rift, fiebre amarilla, zika	Puede confundirse con Ae. albopictus por su morfología similar; el tórax es útil para distinguirlos.
Aedes albopictus	Zika, chikungunya y dengue	Mosquito negro y pequeño (6–9 mm), con una sola línea blanca en la cabeza y el tórax; patas negras con manchas blancas que le dan aspecto de cebra.
Anopheles	Filariasis linfática, malaria, virus O'nyong'nyong	Palpos largos, casi como la probóscide; manchas oscuras y claras alineadas con las venas de las alas.
Culex	Encefalitis japonesa, filariasis linfática, fiebre del Nilo Occidental	Alas con escamas anchas y asimétricas, claras y oscuras entremezcladas; alas moteadas.
Culiseta	Arbovirus, filariasis linfática	Escamas en la parte inferior de la vena subcostal y manchas oscuras marcadas sobre las venas alares.
Aedes japonicus/koreicus	Fiebre del Nilo Occidental, encefalitis japonesa, virus La Crosse, dengue y chikungunya	Varias líneas de escamas doradas en el tórax, dos de ellas muy largas; Ae. japonicus carece de mancha blanca en el último segmento de la tercera pata, mientras que Ae. koreicus tiene un anillo blanco.

Estos rasgos finos (bandas en las patas, escamas en las alas, líneas en el tórax) exigen buena resolución y un encuadre cercano, algo que las fotografías ciudadanas rara vez ofrecen. De ahí que la identificación

automática de especies cercanas, como Ae. aegypti frente a Ae. albopictus, siga siendo un problema abierto, y que la ciencia ciudadana necesite herramientas que filtren y clasifiquen las imágenes recibidas.

3.3 Algoritmos de aprendizaje automático

3.3.1 Redes neuronales convolucionales y ajuste fino

Las redes neuronales convolucionales (CNN) aprenden filtros que detectan patrones locales (bordes, texturas, formas) y los combinan en capas sucesivas hasta obtener representaciones de alto nivel. El aprendizaje por transferencia reutiliza una red entrenada con un conjunto grande (por ejemplo ImageNet) y la adapta a una tarea nueva; el ajuste fino (fine-tuning) continúa el entrenamiento con los datos propios, a veces congelando las primeras capas para reducir el costo y el sobreajuste. En este proyecto se usaron dos arquitecturas:

- **EfficientNet-B0 (Tan y Le, 2019):** red compacta (unos 4 millones de parámetros entrenables) que escala de forma equilibrada profundidad, ancho y resolución.
- **ConvNeXt-Tiny (Liu et al., 2022):** CNN modernizada con ideas de los transformers, con unos 26.6 millones de parámetros entrenables.

3.3.2 Transformers de visión y DINOv2

Los Vision Transformers (ViT; Dosovitskiy et al., 2021) dividen la imagen en parcelas y las procesan con mecanismos de atención. DINOv2 (Oquab et al., 2024) es una familia de ViT entrenada con aprendizaje autosupervisado sobre un conjunto enorme de imágenes, que produce características visuales de propósito general sin necesidad de etiquetas. Con el modelo congelado se extrae un vector de características por imagen (384 dimensiones en la variante ViT-S/14) y sobre él se entrena un clasificador sencillo. Esto reduce de forma drástica el costo de entrenamiento porque las características se calculan una sola vez.

3.3.3 Clasificadores sobre características

- **Regresión logística:** modelo lineal que estima probabilidades por clase; el parámetro C controla la regularización (valores pequeños penalizan más los pesos grandes).
- **Máquina de vectores de soporte (SVM) con núcleo RBF (Cortes y Vapnik, 1995):** busca la frontera que maximiza el margen entre clases y, con el núcleo RBF, puede separar clases no lineales. C controla el compromiso entre margen y errores.
- **Clasificadores jerárquicos:** dividen el problema en niveles. Se evaluaron una variante en cascada (primero se decide entre albopictus, Culex u «otro», y luego se refina) y una variante taxonómica de dos niveles, que explotan que las clases mayoritarias se separan bien y las raras se pueden tratar aparte.

3.3.4 Detección de objetos con YOLOv8

YOLO (You Only Look Once; Redmon et al., 2016) es una familia de detectores de una etapa que predice cajas y clases directamente en un solo paso, con baja latencia. YOLOv8n (Jocher et al., 2023) es la variante más pequeña de YOLOv8; Terven et al. (2023) revisan la evolución de la familia. Se inicializó con pesos

preentrenados en COCO y se entrenó con una sola clase («mosquito»), de modo que el detector solo localiza y la identificación de la especie queda a cargo del clasificador.

3.3.5 Manejo del desbalance de clases

Cuando unas clases tienen muchas más muestras que otras, los modelos tienden a favorecer a las mayoritarias. Buda et al. (2018) comparan estrategias y encuentran que el sobremuestreo hasta equilibrar las clases es eficaz en redes convolucionales. En este proyecto se usó sobremuestreo con WeightedRandomSampler, aumento de datos y label smoothing en las CNN, y ponderación de clases balanceada (class_weight='balanced') en los modelos sobre DINOv2.

3.3.6 Métricas de evaluación

- **Exactitud:** proporción de aciertos; engañosa con clases desbalanceadas.
- **Exactitud balanceada:** promedio del recall de cada clase.
- **F1 macro:** promedio simple del F1 (media armónica de precisión y recall) de cada clase; pesa igual a todas las clases y es la métrica oficial del reto.
- **mAP@50 y mAP@50-95:** precisión media del detector con umbrales de IoU de 0.5 y promediando de 0.5 a 0.95.
- **Matriz de confusión:** tabla que cruza la clase real con la predicha y muestra qué clases se confunden.

3.4 Antecedentes: estudios previos

La mayoría de los trabajos sobre identificación de mosquitos usa CNN, casi siempre con transferencia de aprendizaje. Motta et al. (2019) entrenaron LeNet, AlexNet y GoogLeNet con 4,056 imágenes de campo de tres especies y obtuvieron exactitudes de validación de 57.5 %, 74.7 % y 83.9 %, lo que muestra que la arquitectura importa. Motta et al. (2020) probaron Xception y DenseNet con búsqueda de hiperparámetros y alcanzaron exactitudes balanceadas de 93.5 % (mosquitos frente a otros insectos) y 97.3 % (Aedes frente a Culex). Park et al. (2020) visualizaron las regiones que usa la red y concluyeron que se apoya en rasgos morfológicos parecidos a los de los expertos. Ong et al. (2021) separaron Ae. aegypti de Ae. albopictus con MobileNet (más de 98 %), aunque con fotos tomadas en un dispositivo estandarizado. Goodwin et al. (2021) usaron CNN con un conjunto multinivel que además detecta especies nuevas.

Tabla 2. *Estudios previos de clasificación de mosquitos con aprendizaje profundo.*

Estudio	Datos	Enfoque	Resultado reportado
Motta et al. (2019)	4,056 imágenes; 3 especies	LeNet, AlexNet, GoogLeNet	57.5 %, 74.7 % y 83.9 % de exactitud
Motta et al. (2020)	7,561 mosquitos y 1,187 otros insectos	Xception y DenseNet con búsqueda de hiperparámetros	Exactitud balanceada de 93.5 % y 97.3 %
Park et al. (2020)	Imágenes de mosquitos vectores	CNN profundas y análisis de regiones discriminativas	Rasgos aprendidos similares a los de los expertos
Kittichai et al. (2021)	10,564 imágenes; 15 clases	YOLO v2 y v3, una y dos etapas	Dos etapas superó a una etapa

Estudio	Datos	Enfoque	Resultado reportado
Ong et al. (2021)	4,120 imágenes; 2 especies	MobileNet con transferencia	Más de 98 %, equivalente a expertos

Las cifras son las que reportan los autores y no son comparables entre sí porque los datos y las condiciones de captura son distintos.

En detección, Kittichai et al. (2021) compararon YOLO de una etapa (localiza y clasifica a la vez) contra dos etapas encadenadas (una primera red detecta el mosquito, se recorta esa región y otra red lo identifica). Con todas las variantes probadas la versión de dos etapas rindió mejor, y el desempeño por clase seguía de cerca el número de imágenes de entrenamiento; sugieren reunir más de unas 300 imágenes por clase. Pataki et al. (2021) entrenaron un modelo con la base curada de Mosquito Alert, la misma fuente de este proyecto, para reconocer al mosquito tigre en fotos ciudadanas. Duan et al. (2026) proponen LW-YOLO, una variante ligera de YOLOv8 para detectar mosquitos.

Sobre transferencia de aprendizaje, Oquab et al. (2024) muestran que DINOv2 produce características de uso general que sirven sin ajuste fino. Sobre desbalance, Buda et al. (2018) encuentran que el sobremuestreo fue el mejor método en casi todos los escenarios y que, a diferencia de los modelos clásicos, no provoca sobreajuste en CNN. Sobre clasificación jerárquica, Silla y Freitas (2011) ofrecen un marco unificador de los enfoques existentes; el riesgo de separar por niveles es que un error en el nivel superior ya no se puede corregir en el inferior.

De esta revisión se desprenden tres brechas: la mayoría de los estudios usa imágenes de laboratorio o de dispositivos controlados y pocos trabajan con fotografías ciudadanas; los conjuntos reportados suelen estar balanceados o tener decenas o cientos de ejemplos por clase, con un desbalance menos severo que el de este reto; y en las fuentes revisadas no se encontró una comparación directa entre clasificación plana y jerárquica en mosquitos, por lo que esa comparación es una contribución propia del proyecto.

Nota sobre el alcance: el proyecto trabaja únicamente con imágenes; por ello el marco teórico no incluye las secciones de procesamiento de lenguaje natural ni de sistemas de recomendación, que no aplican.

4. Metodología

4.1 Pasos seguidos por el grupo

- Obtención del conjunto de datos del reto (train.csv e imágenes) y revisión bibliográfica.
- Limpieza de cajas inválidas y análisis exploratorio.
- Partición estratificada 70/15/15 y generación de recortes de 256 × 256 px y del conjunto en formato YOLO.
- Extracción de características con DINOv2 y entrenamiento de los clasificadores sobre ellas (CPU).
- Ajuste fino de EfficientNet-B0 y ConvNeXt-Tiny, previa medición del rendimiento de la GPU.
- Entrenamiento del detector YOLOv8n.
- Evaluación en el conjunto de prueba, análisis de errores y combinación de modelos.

- Desarrollo de la aplicación en Streamlit y verificación de que reproduce las predicciones guardadas.

4.2 Selección de los conjuntos de entrenamiento, validación y prueba

Se hizo una partición estratificada por clase en 70 % entrenamiento, 15 % validación y 15 % prueba, con semilla 42 para que sea reproducible. La estratificación garantiza que las clases raras aparezcan en las tres particiones (por ejemplo, aegypti tiene 33, 7 y 7 imágenes). La partición se realizó antes de cualquier sobremuestreo para no filtrar copias de una misma imagen entre particiones. La validación se usó para elegir hiperparámetros y la mejor época; la prueba se reservó exclusivamente para las cifras finales.

4.3 Selección de los algoritmos y razones

- **DINOv2 congelado + clasificadores simples:** con 4 GB de memoria de GPU, entrenar redes grandes es costoso. Extraer las características una sola vez permite probar varios clasificadores en minutos. Se incluyeron regresión logística (línea base lineal) y SVM RBF (frontera no lineal).
- **Variantes jerárquicas:** se compararon dos esquemas con el mismo backbone, con la hipótesis de que separar por niveles ayuda a las clases raras. La cascada decide primero entre albopictus, Culex y «otro», y solo en este último caso un segundo modelo separa Culiseta, japonicus/koreicus, Anopheles y aegypti. La variante taxonómica usa en el primer nivel los géneros {Aedes, Culex, Culiseta, Anopheles} y en el segundo separa las especies de Aedes, de modo que las clases más parecidas quedan juntas. Se anticipó el riesgo de propagación de errores: lo que se equivoque el primer nivel no se corrige en el segundo (Silla y Freitas, 2011).
- **EfficientNet-B0 y ConvNeXt-Tiny con ajuste fino:** representan el enfoque estándar de aprendizaje profundo; EfficientNet-B0 por su bajo costo y ConvNeXt-Tiny por su capacidad mayor.
- **YOLOv8n:** detector pequeño y rápido, adecuado para el hardware disponible y para integrarse en una app interactiva.
- **Detector + clasificador en dos etapas:** se planteó en la revisión bibliográfica; recortar al mosquito elimina el fondo y concentra al clasificador en el sujeto.

4.4 Herramientas utilizadas

Recursos de cómputo

Todo el entrenamiento se realizó de forma local (sin Colab) en una laptop con GPU NVIDIA GeForce GTX 1650 de 4 GB, 16 GB de RAM y procesador i5 de 8 hilos. Dado el límite de memoria, se ejecuta una sola tarea de GPU a la vez y los modelos sobre DINOv2 se entrenan en CPU en paralelo.

Lenguajes, bibliotecas y paquetes

Tabla 3. *Software utilizado (Python 3.11.17).*

Categoría	Biblioteca (versión)
Aprendizaje profundo	torch 2.14.1+cu126, torchvision 0.29.1+cu126, timm 1.0.30

Categoría	Biblioteca (versión)
Detección	ultralytics 8.4.175 (YOLOv8)
Aprendizaje clásico	scikit-learn 1.9.1
Datos	numpy 2.4.6, pandas 3.0.6
Aplicación y gráficas	streamlit 1.65.0, plotly 7.1.0

4.5 Del plan preliminar a la implementación

En la revisión bibliográfica el grupo planteó una selección preliminar de algoritmos, sujeta a confirmación con el análisis exploratorio y los primeros experimentos. La siguiente lista resume qué se mantuvo y qué cambió:

- **Detector:** se planteó YOLO en variantes pequeñas (n y s) con una sola clase; se entrenó YOLOv8n, que cabe en la GPU de 4 GB.
- **Clasificadores:** se mantuvieron los tres candidatos (EfficientNet-B0 como línea base, ConvNeXt-Tiny como modelo principal y DINOv2 congelado con clasificador lineal) y se añadió la SVM con núcleo RBF sobre DINOv2.
- **Variantes plana y jerárquicas:** se compararon las tres propuestas con el mismo backbone (plana, taxonómica y cascada); no se implementó la versión con backbone compartido y dos cabezas.
- **Desbalance:** se aplicaron sobremuestreo hasta clases equiprobables, aumento de datos, ponderación de clases (modelos sobre DINOv2) y label smoothing (CNN). La variante con focal loss y el ajuste final de umbrales para optimizar el F1 macro no aparecen en los resultados finales.
- **Cómputo:** se previó pasar ConvNeXt-Tiny a Google Colab si era necesario; finalmente todo se entrenó en local, reduciendo la resolución a 176 px y congelando las primeras etapas.
- **Evaluación:** se planteó evaluar con los recortes del detector; las métricas de clasificación reportadas se calcularon sobre recortes centrados en la caja anotada, y el detector se evaluó por separado (véanse las limitaciones en las conclusiones).

5. Resultados y análisis de resultados

5.1 Características del conjunto de datos original

El conjunto original (train.csv del MosquitoAlert Challenge 2023) contiene 10,357 imágenes con una caja delimitadora por imagen, anotadas por entomólogos, y 8 variables. La página del reto reporta 8,025 imágenes, pero el archivo descargado contiene 10,357; las imágenes ocupan aproximadamente 9 GB. No hay valores faltantes en ninguna columna ni nombres de archivo duplicados, de modo que cada fila corresponde a una fotografía distinta.

Tabla 4. *Variables del conjunto original.*

Variable	Tipo	Descripción
img_fName	texto	Identificador (nombre de archivo) de la imagen
img_w, img_h	entero	Ancho y alto de la fotografía completa, en píxeles
bbx_xtl, bbx_ytl	entero	Esquina superior izquierda de la caja delimitadora
bbx_xbr, bbx_ybr	entero	Esquina inferior derecha de la caja delimitadora
class_label	categórica	Especie o género (6 clases)

5.2 Limpieza y preprocesamiento

Se verificó que no faltara ninguna imagen referenciada en disco y se descartaron 3 filas con cajas geométricamente inválidas (xbr ≤ xtl, ybr ≤ ytl o fuera de la imagen), quedando 10,354 imágenes. Luego se aplicó el siguiente preprocesamiento (src/prepare_data.py):

- **Recorte para clasificadores:** recorte cuadrado centrado en la caja anotada con 15 % de margen, redimensionado a 256 × 256 px. La aplicación usa exactamente el mismo recorte sobre la caja del detector.
- **Datos para el detector:** imagen completa reescalada a 640 px en el lado mayor, con etiqueta YOLO de una sola clase.
- **Decodificación JPEG a media resolución (draft):** acelera aproximadamente 4 veces el preprocesamiento de los 9 GB de imágenes.
- **CNN:** aumento de datos (RandomResizedCrop 0.6–1.0, volteos horizontal y vertical, rotación ±20°, ColorJitter) y sobremuestreo con WeightedRandomSampler hasta clases equiprobables; evaluación con recorte central de 224 px.
- **DINOv2:** características congeladas de 384 dimensiones (ViT-S/14, entrada de 224 px) extraídas una sola vez, con estandarización y class_weight balanceado.

5.3 Análisis exploratorio

El análisis exploratorio se realizó en el cuaderno 01_EDA_mosquito_species.ipynb sobre el archivo original (10,357 filas). A partir de las coordenadas de la caja se derivaron variables que describen el tamaño del mosquito: ancho y alto de la caja, su área, la proporción del área de la caja respecto a la imagen (bbx_area_ratio) y la relación de aspecto de la imagen (ancho/alto).

5.3.1 Variables cuantitativas

Tabla 5. *Estadística descriptiva de las variables cuantitativas (n = 10,357, antes de la limpieza).*

Variable	Media	Mediana	Mínimo	Máximo
img_w (px)	2,040	1,844	139	9,280
img_h (px)	2,470	2,208	106	12,000
bbx_width (px)	663	477	1	4,064
bbx_height (px)	678	487	48	5,580

Variable	Media	Mediana	Mínimo	Máximo
bbx_area_ratio	0.131	0.073	0.000005	0.957
aspect_ratio	0.875	0.750	0.360	2.878

Las dimensiones de las fotos varían enormemente (de 139 a 9,280 px de ancho y de 106 a 12,000 px de alto). Los histogramas (Figura 1) son multimodales, con picos alrededor de 800–1,000 px y de 3,000–4,000 px, consistente con fotos tomadas con distintos modelos de celular o cámara, típico de un conjunto de ciencia ciudadana. En promedio el mosquito ocupa el 13.1 % del área de la foto, pero la mediana es solo 7.3 %: la distribución tiene una cola larga hacia la derecha, es decir, en la mayoría de las fotos el insecto es un objeto pequeño y solo un grupo minoritario (probablemente tomas macro) eleva el promedio. La relación de aspecto tiene mediana 0.75, lo que indica que la mayoría de las fotos se tomó en formato vertical.

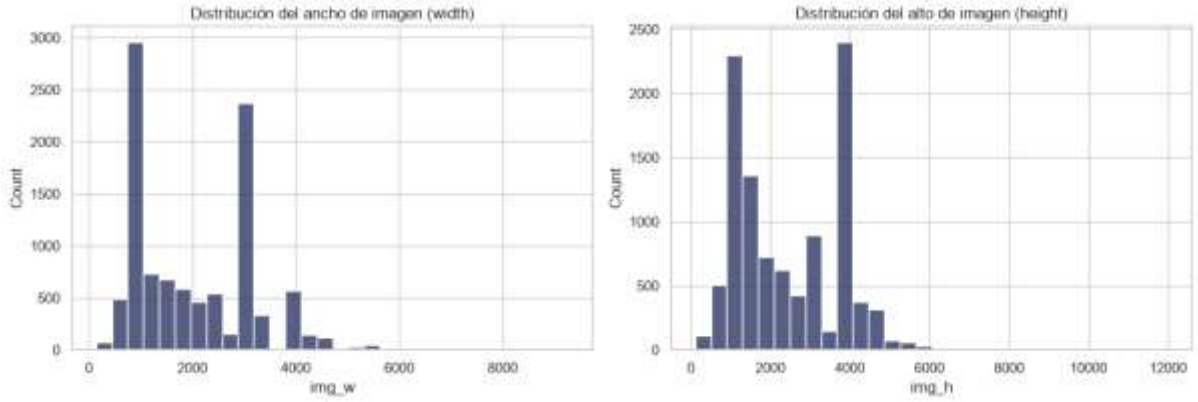


Figura 1. *Distribución del ancho y el alto de las imágenes.*

5.3.2 Tamaño relativo del mosquito por clase

Tabla 6. *Proporción del área de la caja respecto a la imagen (bbx_area_ratio) y relación de aspecto, por clase.*

Clase	Media	Mediana	Desv. est.	Aspecto (media)
aegypti	0.285	0.281	0.262	0.915
albopictus	0.128	0.070	0.147	0.907
anopheles	0.154	0.102	0.151	0.752
culex	0.121	0.066	0.137	0.850
culiseta	0.190	0.121	0.180	0.847
japonicus/koreicus	0.178	0.123	0.168	0.865

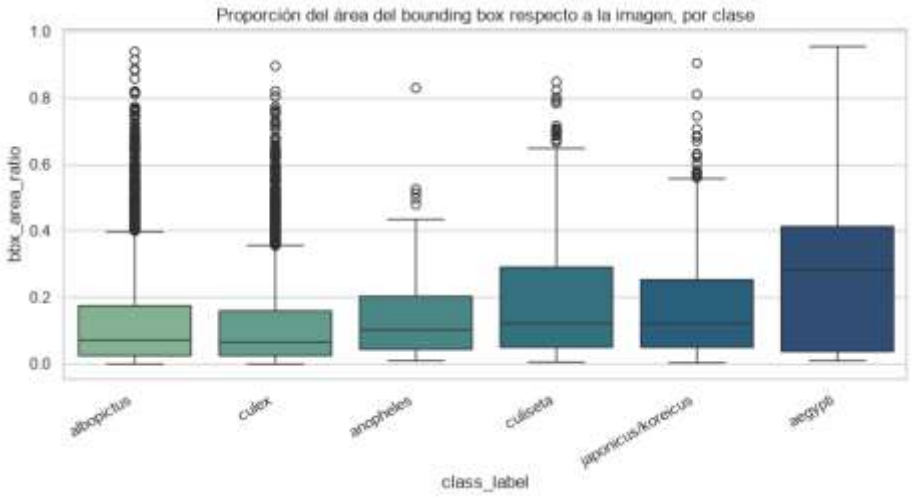


Figura 2. *Proporción del área de la caja respecto a la imagen, por clase.*

Aegypti tiene la proporción media y mediana más altas (0.285 y 0.281), muy por encima del resto: en sus fotos el mosquito ocupa una porción mucho mayor del encuadre, posiblemente porque son tomas más cercanas, varias con lupa o microscopio (Figura 7). En el otro extremo, albopictus y Culex, las clases mayoritarias, tienen las proporciones más bajas (medianas de 0.07 y 0.066) y una gran cantidad de valores atípicos hacia arriba, simplemente porque tienen muchas más observaciones. En todas las clases la media es mayor que la mediana, lo que confirma la asimetría.

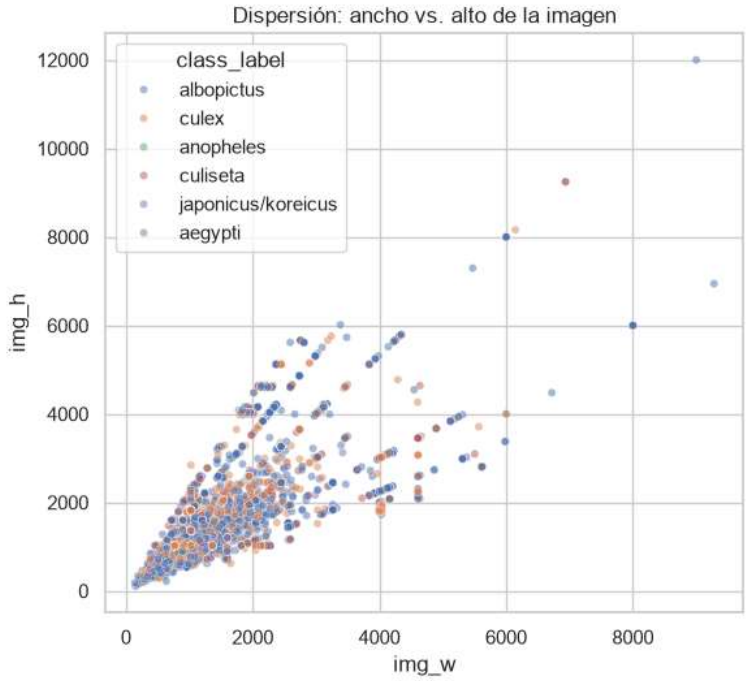


Figura 3. *Dispersión del ancho frente al alto de las imágenes, por clase.*

El diagrama de dispersión (Figura 3) muestra una relación positiva fuerte y esperable entre ancho y alto, con la nube concentrada por debajo de 4,000 × 4,000 px y algunos puntos aislados hasta 9,000 × 12,000

px. Las clases aparecen completamente mezcladas, es decir, el tamaño de la imagen depende del dispositivo y no de la especie fotografiada.

5.3.3 Distribución de clases

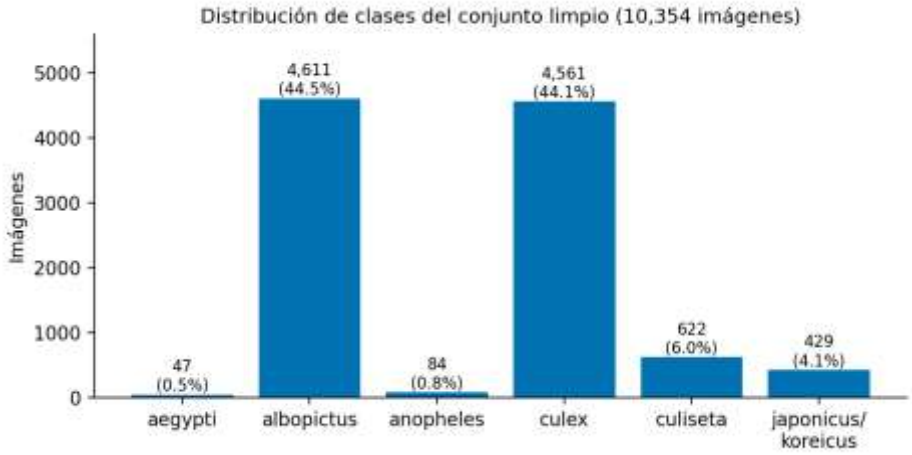


Figura 4. *Distribución de clases del conjunto limpio.*

Tabla 7. *Partición estratificada por clase.*

Clase	Train	Val	Test	Total	% del total
aegypti	33	7	7	47	0.45
albopictus	3,227	692	692	4,611	44.53
anopheles	58	13	13	84	0.81
culex	3,193	684	684	4,561	44.05
culiseta	436	93	93	622	6.01
japonicus/koreicus	301	64	64	429	4.14
Total	**7,248**	**1,553**	**1,553**	**10,354**	**100.00**

Hay un desbalance de clases severo (Figura 4): albopictus (44.5 %) y Culex (44.1 %) concentran el 88.6 % de las observaciones; Culiseta (6.0 %) y japonicus/koreicus (4.1 %) son minoritarias moderadas, y Anopheles (0.8 %) y aegypti (0.45 %) son extremadamente raras. La clase mayoritaria tiene 98 veces más imágenes que la minoritaria. Esto tiene consecuencias directas para el modelado: (1) la exactitud global es engañosa, pues un modelo que siempre predijera albopictus o Culex acertaría casi 9 de cada 10 veces sin aprender nada de las otras cuatro clases, por lo que se adoptó el F1 macro; (2) las clases raras tienen muy pocas imágenes de prueba (7 para aegypti, 13 para Anopheles), de modo que sus métricas tienen alta varianza; y (3) se requieren técnicas explícitas de balanceo, tanto más cuanto que las clases raras son epidemiológicamente relevantes (aegypti es vector de dengue, zika y chikungunya).

5.3.4 Cruce de variables

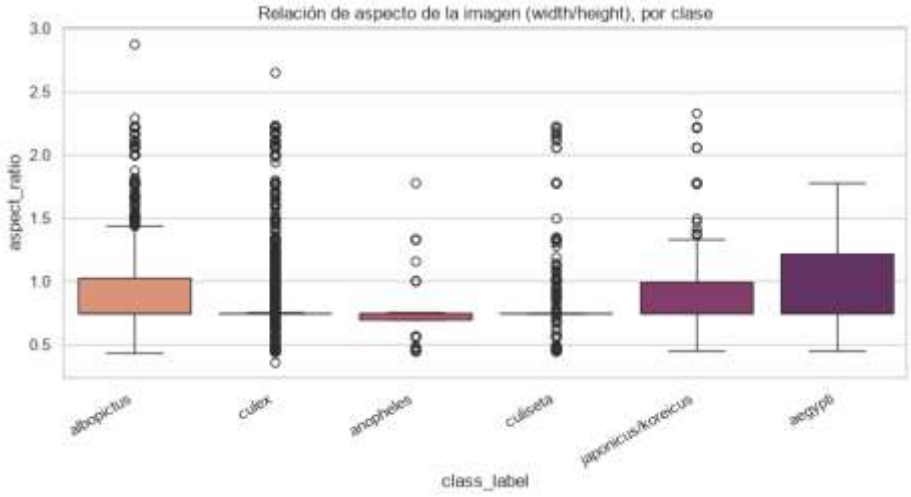


Figura 5. *Relación de aspecto de la imagen, por clase.*

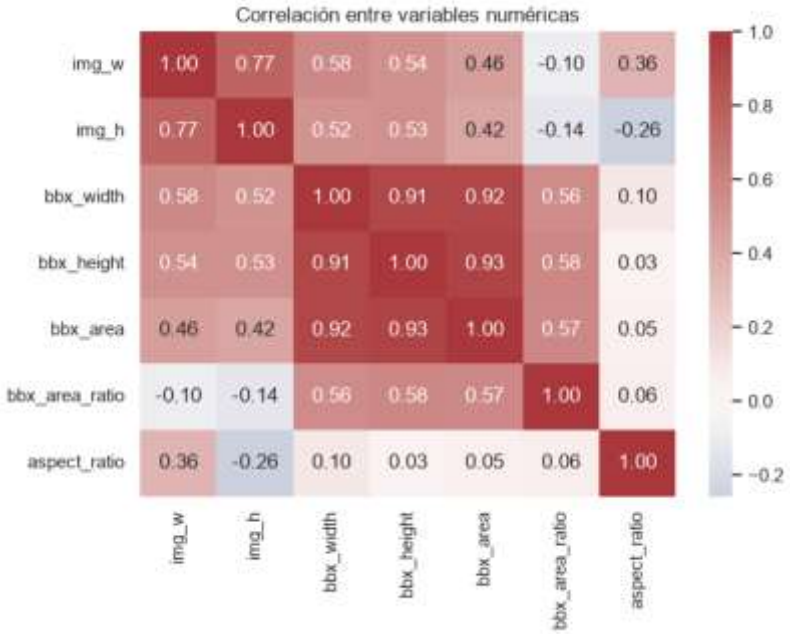


Figura 6. *Correlación entre las variables numéricas derivadas.*

La matriz de correlación (Figura 6) muestra alta redundancia entre las variables derivadas de la caja: el ancho y el alto correlacionan 0.91 entre sí y con el área (0.92 y 0.93), lo esperable porque el área es su producto, por lo que no conviene usarlas juntas como variables independientes. En cambio, bbx_area_ratio correlaciona solo moderadamente con ellas (0.56 a 0.58) y de forma débil con las dimensiones de la imagen completa (−0.10 con el ancho y −0.14 con el alto): normalizar por el área de la imagen aporta información distinta al tamaño absoluto de la caja.

En la relación de aspecto por clase (Figura 5), Culex, Anopheles y Culiseta tienen cajas angostas centradas en 0.75 con valores atípicos hacia arriba, mientras que albopictus, japonicus/koreicus y sobre todo aegypti muestran cajas más anchas, con bigotes que llegan hasta cerca de 1.2–1.8. Esto sugiere cierta asociación

entre la clase y el formato de la imagen, que podría deberse a que las clases más raras se capturaron con equipos distintos (por ejemplo, a través de una lupa o un microscopio).

5.3.5 Valores atípicos y faltantes

El método del rango intercuartílico (IQR) detectó 729 valores atípicos en bbx_area_ratio (7.0 % del total) y 698 en aspect_ratio (6.7 %). Al ser proporciones no marginales y corresponder a variabilidad real de la captura ciudadana (fotos macro en un caso, formatos de imagen poco comunes en el otro) y no a errores de medición, se decidió conservarlos y, de ser necesario, tratarlos con técnicas robustas en lugar de descartar filas. Tras agregar las variables derivadas se verificó que no hay valores nulos en ninguna de las 14 columnas, por lo que no se requirió imputación.

5.3.6 Inspección visual de las imágenes

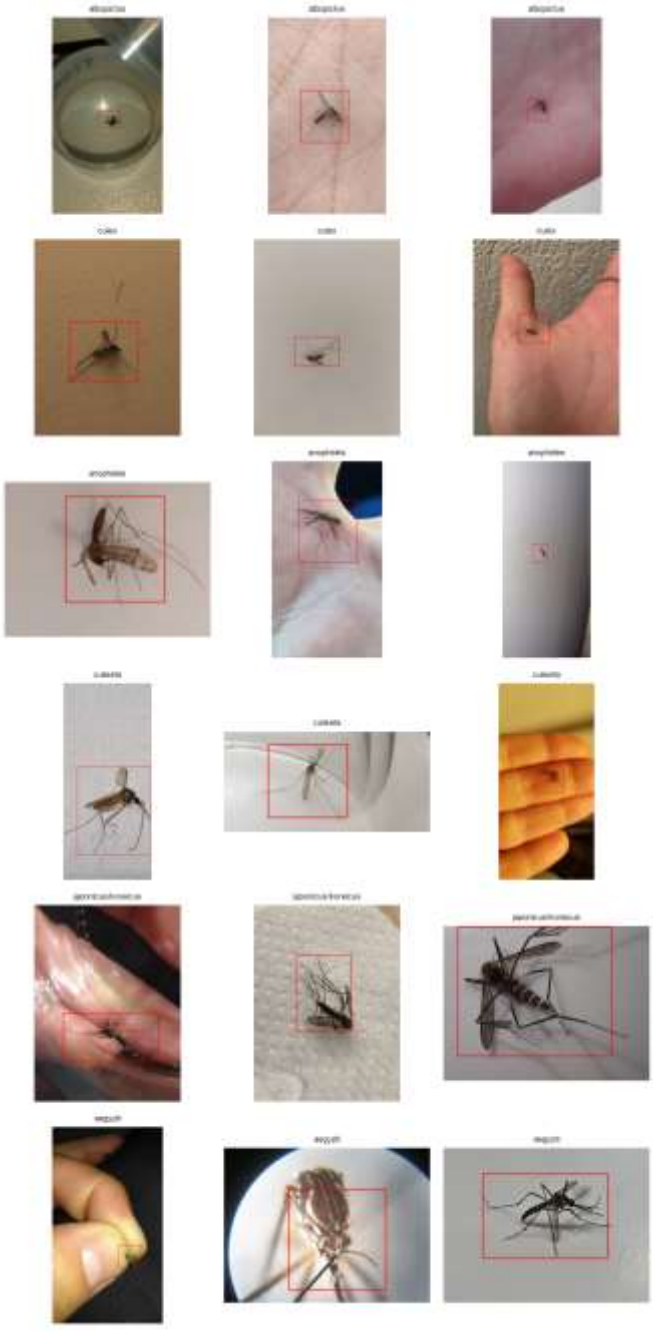


Figura 7. *Ejemplos de imágenes por clase con su caja delimitadora (en rojo).*

La inspección visual (Figura 7) confirma que varias clases son difíciles de distinguir a simple vista: Culex, Anopheles y Culiseta comparten un cuerpo alargado y patas finas muy similares, y solo se diferencian por detalles pequeños (bandas en las patas, postura de las alas en reposo) que exigen buena resolución y encuadre cercano. En la mayoría de las fotos el mosquito ocupa poco espacio, excepto en varias de aegypti y japonicus/koreicus tomadas con lupa o desde muy cerca. El fondo y la iluminación varían muchísimo (piel de la mano, tela, mesas, papel, cajas de Petri, luz natural o flash), lo que respalda la necesidad de un

aumento de datos robusto para que el modelo no asocie el fondo con la clase en lugar de los rasgos del insecto.

5.3.7 Hallazgos del análisis exploratorio y decisiones derivadas

- Desbalance de 98:1: F1 macro como métrica principal, partición estratificada antes del sobremuestreo, ponderación de clases y sobremuestreo.
- Mosquito pequeño en la mayoría de las fotos: detección explícita con YOLOv8n y recorte centrado en la caja con 15 % de margen para que el clasificador reciba al insecto y no el fondo.
- Dimensiones y formatos muy variables: redimensionamiento a un tamaño fijo y decodificación a media resolución para procesar 9 GB de imágenes.
- Fondos e iluminación muy diversos: aumento de datos con recortes aleatorios, volteos, rotaciones y cambios de color.
- Valores atípicos legítimos: se conservaron todas las observaciones; solo se eliminaron las 3 cajas geométricamente inválidas.

5.4 Ajuste de parámetros de cada algoritmo

5.4.1 Medición de rendimiento de la GPU

Antes de entrenar las CNN se midió el rendimiento de entrenamiento en la GTX 1650 (Figura 8). Con EfficientNet-B0, usar precisión mixta automática (AMP) elevó el rendimiento ligeramente (116 a 121 img/s), pero el formato de memoria channels_last lo redujo drásticamente a 16 img/s. Por eso se entrenó con AMP en formato NCHW. ConvNeXt-Tiny no superó 13 img/s a 224 px; para hacerlo viable se redujo la resolución a 176 px y se congelaron el stem y las dos primeras etapas.

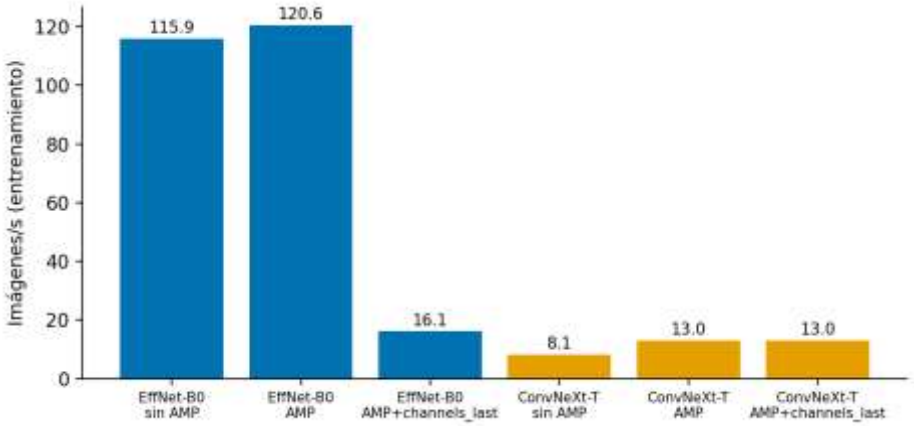


Figura 8. *Rendimiento de entrenamiento en la GTX 1650 según configuración.*

5.4.2 Clasificadores sobre DINOv2

Se buscó el parámetro C en validación usando F1 macro. Para la regresión logística, el mejor valor fue C = 0.1; para la SVM, C = 10 (el máximo explorado, con mejoras marginales respecto a C = 3). Los modelos jerárquicos reutilizaron el C de la regresión logística plana.

Tabla 8. *Búsqueda de C (F1 macro en validación; se indica el elegido).*

Modelo	Valores probados y F1 macro
dinov2_logreg	C=0.01 → 0.620 · C=0.1 → 0.634 (elegido) · C=1 → 0.623 · C=10 → 0.581
dinov2_svm	C=1 → 0.684 · C=3 → 0.691 · C=10 → 0.692 (elegido)

5.4.3 CNN con ajuste fino

Tabla 9. *Configuración elegida para las CNN.*

Parámetro	EfficientNet-B0	ConvNeXt-Tiny
Épocas	6	3
Tamaño de lote	32	32
Tasa de aprendizaje	0.0005	0.0003
Resolución	224 px	176 px
Capas congeladas	ninguna	stem + 2 primeras etapas
Optimizador / planificador	AdamW + OneCycle	AdamW + OneCycle
Label smoothing	0.1	0.1
Parámetros entrenables	4,015,234	26,589,702
Tiempo de entrenamiento	8.7 min	17.6 min

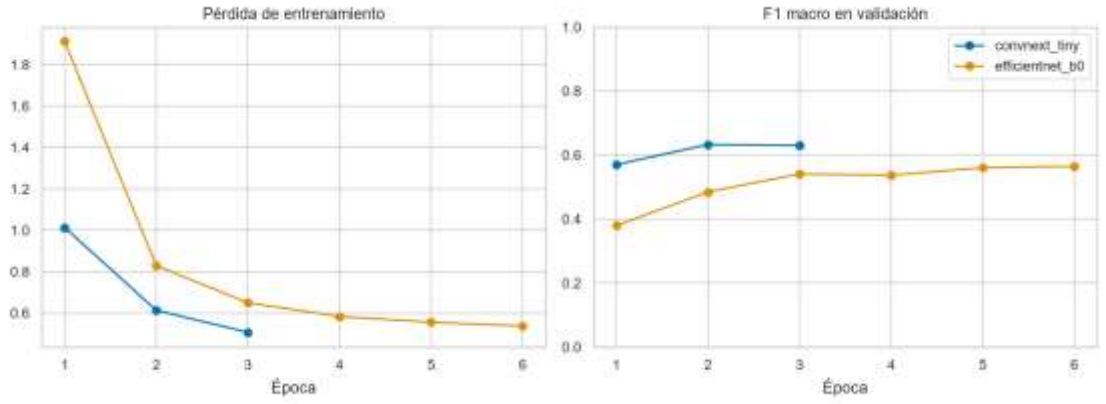


Figura 9. *Curvas de entrenamiento de las CNN (se conserva la época con mejor F1 en validación).*

ConvNeXt-Tiny alcanzó su mejor F1 de validación (0.633) en la época 2, mientras que EfficientNet-B0 seguía mejorando lentamente en la época 6 (0.565), lo que sugiere que con más épocas podría mejorar, aunque el tiempo de cómputo disponible lo limitó.

5.4.4 Detector YOLOv8n

El detector se inicializó con YOLOv8n preentrenado en COCO y se entrenó con imágenes de 512 px, lote de 16, AMP y paciencia de 5 épocas. Se fijó un tope de 12 épocas y de 0.3 h, por lo que se completaron 6 épocas en 20 minutos. Las métricas de validación aún mejoraban al finalizar (Figura 10), señal de que el detector no había convergido del todo.

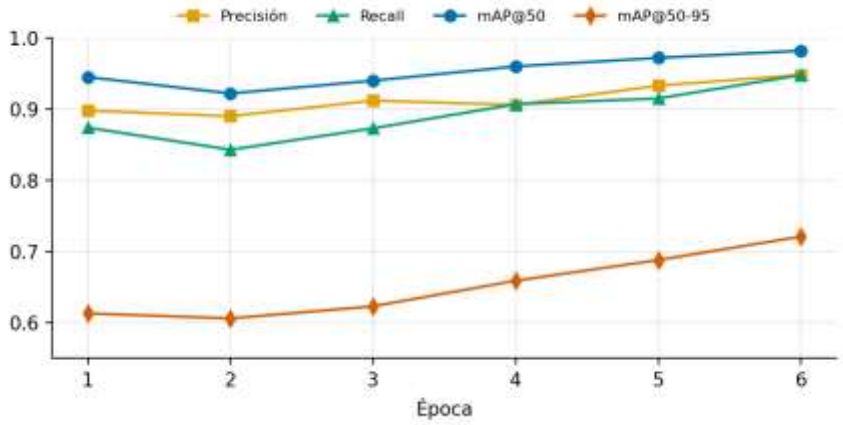


Figura 10. *Métricas de validación del detector por época.*

5.5 Comparación de los algoritmos

Tabla 10. *Resultados en el conjunto de prueba (1,553 imágenes), ordenado por F1 macro.*

Modelo	F1 macro	Exactitud balanceada	Exactitud	Entrenamiento (min)	Inferencia (ms/img)
DINOv2 + SVM	**0.664**	**0.618**	**0.940**	**1.9**	**37.3**
Jerárquico en cascada	0.662	0.689	0.907	0.2	36.0
DINOv2 + Reg. logística	0.630	0.707	0.900	0.1	36.0
Jerárquico taxonómico	0.623	0.684	0.896	0.3	36.0
ConvNeXt-Tiny	0.565	0.606	0.892	17.6	19.9
EfficientNet-B0	0.540	0.568	0.876	8.7	12.3

Los tiempos de entrenamiento de los modelos DINOv2 excluyen la extracción de características (unos 6 minutos, una sola vez). El tiempo de inferencia de esos modelos incluye la extracción de características en GPU.

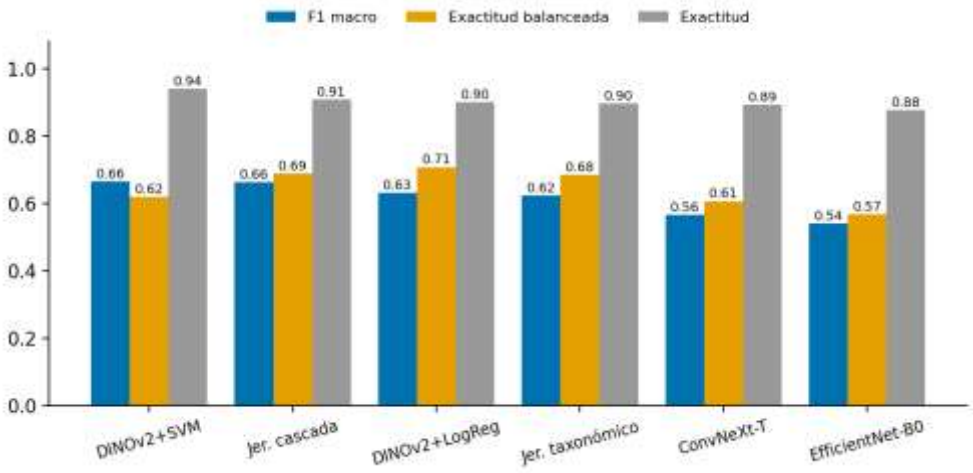


Figura 11. *Comparación de métricas globales de los seis modelos en prueba.*

La SVM sobre DINOv2 obtuvo el mejor F1 macro (0.664) y la mayor exactitud (0.940), y la cascada jerárquica quedó prácticamente empatada (0.662). Las características congeladas de DINOv2 con un clasificador simple superaron a las dos CNN con ajuste fino, entrenando en una fracción del tiempo (menos de 2 minutos frente a 9–18 minutos). Hay que matizar que a las CNN se les dio un presupuesto de cómputo limitado (pocas épocas, resolución reducida en ConvNeXt), por lo que la comparación refleja lo alcanzable en este hardware y no el máximo de cada arquitectura.

Nótese que el orden cambia según la métrica: la regresión logística tiene la mayor exactitud balanceada (0.707) pero un F1 macro menor, porque recupera más ejemplares de las clases raras a costa de precisión (más falsos positivos). La exactitud global (0.88–0.94) es alta para todos porque las dos clases mayoritarias dominan.

5.5.1 Desempeño por clase

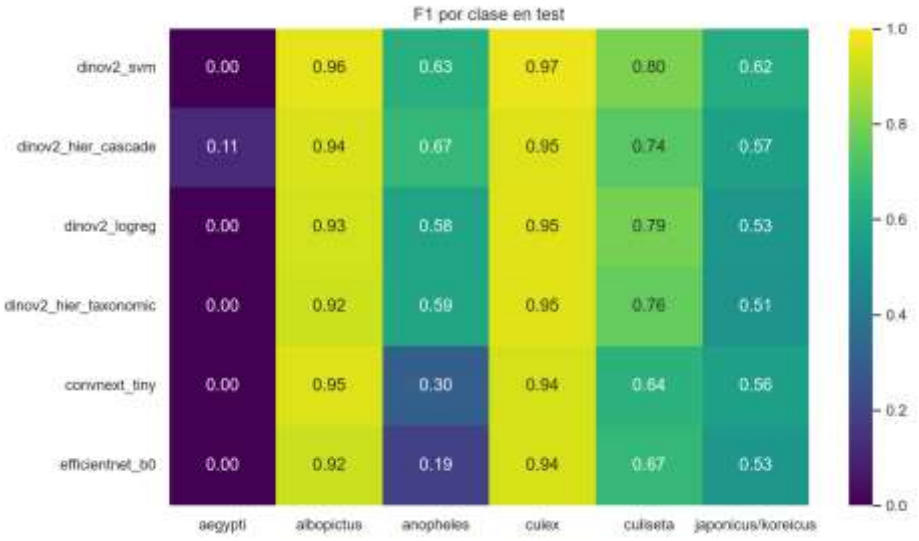


Figura 12. *F1 por clase y modelo en el conjunto de prueba.*

Tabla 11. *Recall por clase en prueba.*

Modelo	aegypti	albopictus	anopheles	culex	culiseta	japon./kor.
DINOv2 + SVM	0.000	0.980	0.462	0.982	0.753	0.531
Jer. cascada	0.143	0.926	0.692	0.942	0.774	0.656
DINOv2 + LogReg	0.000	0.905	0.846	0.931	0.839	0.719
Jer. taxonómico	0.000	0.893	0.769	0.940	0.828	0.672
ConvNeXt-Tiny	0.000	0.931	0.231	0.898	0.828	0.750
EfficientNet-B0	0.000	0.883	0.154	0.927	0.710	0.734

Todos los modelos reconocen muy bien albopictus y Culex (F1 de 0.92 a 0.97). Culiseta alcanza 0.64–0.81. Las clases difíciles son Anopheles (F1 de 0.19 a 0.67), japonicus/koreicus (0.51–0.62) y sobre todo Ae. aegypti, con F1 de 0 en cinco de los seis modelos; solo la cascada jerárquica reconoce 1 de las 7 imágenes

de prueba (F1 de 0.111). Estas clases tienen pocas imágenes de entrenamiento (33 y 58) y pocas de prueba, por lo que cada acierto o error mueve mucho la métrica.

5.5.2 Análisis de errores

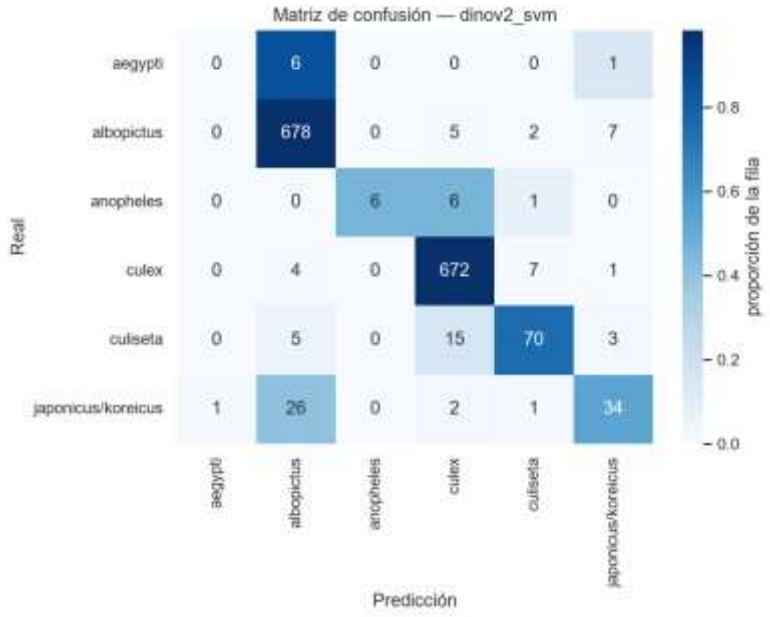


Figura 13. *Matriz de confusión de DINOv2 + SVM (conteos por celda; el color indica la proporción de la fila).*

En la matriz del mejor modelo se observan tres patrones: (1) 6 de las 7 imágenes de aegypti se predicen como albopictus, lo que es coherente con que ambas especies del género Aedes son morfológicamente similares; (2) 26 de las 64 imágenes de japonicus/koreicus se predicen como albopictus, otra confusión dentro del género Aedes; y (3) la mitad de los Anopheles (6 de 13) se confunden con Culex, y 15 de las 93 Culiseta se predicen como Culex. Es decir, los errores se concentran entre especies cercanas y hacia la clase mayoritaria, lo esperable cuando el clasificador ve muy pocos ejemplares de las clases raras.

Los modelos con sobremuestreo o ponderación balanceada (regresión logística, jerárquicos, CNN) corrigen parte de ese sesgo: aumentan el recall de las clases raras pero a cambio predicen más falsos positivos, sobre todo albopictus clasificado como japonicus/koreicus (52 casos con la regresión logística frente a 7 con la SVM).

La revisión bibliográfica anticipó que la cascada podría perjudicar a aegypti por propagación de errores, ya que esta especie se parece a albopictus y el primer nivel la enviaría a una clase de la que el segundo nivel no la rescata. En los resultados esto no se confirmó para la cascada: fue la única variante que recuperó una de las 7 imágenes de aegypti y la que más mejoró a Anopheles frente al modelo plano con el mismo backbone (F1 de 0.667 frente a 0.579 de la regresión logística). Con tan pocas imágenes de prueba la evidencia es débil, y a cambio la cascada pierde exactitud global (0.907 frente a 0.940 de la SVM).

5.5.3 Costo computacional frente a desempeño

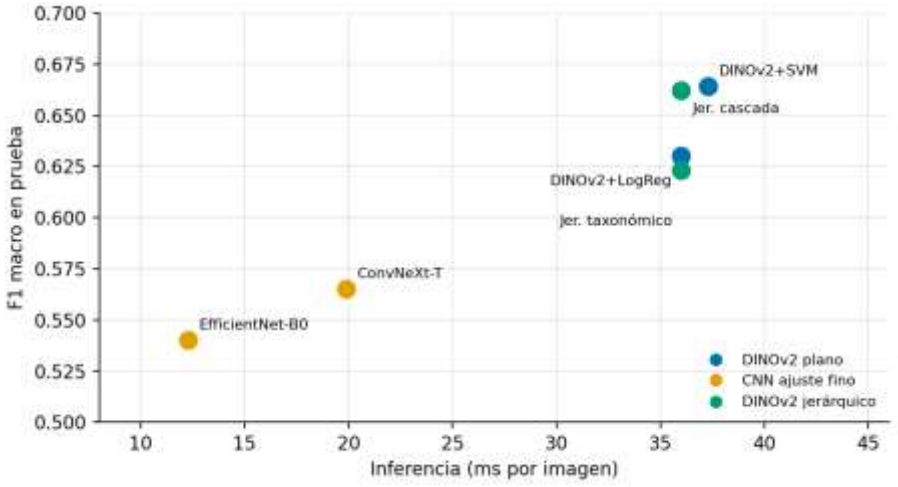


Figura 14. *F1 macro en prueba frente al tiempo de inferencia por imagen.*

Los modelos sobre DINOv2 son los más costosos en inferencia (unos 36–37 ms por imagen, dominados por la extracción de características) pero los más baratos de entrenar. Las CNN infieren más rápido (12–20 ms) pero requieren entre 9 y 18 minutos de entrenamiento y rindieron menos. La SVM sobre DINOv2 ofrece el mejor equilibrio para esta aplicación, donde la latencia de decenas de milisegundos es aceptable.

5.5.4 Combinación y acuerdo entre modelos

Como análisis complementario, se promediaron las probabilidades de los seis modelos sin seleccionar miembros con el conjunto de prueba. El resultado fue un F1 macro de 0.672 (exactitud balanceada 0.691, exactitud 0.921), ligeramente por encima del mejor modelo individual (0.664). Por clase, el promedio obtuvo F1 de 0.00 (aegypti), 0.95 (albopictus), 0.67 (anopheles), 0.96 (culex), 0.80 (culiseta) y 0.66 (japonicus/koreicus).

Además, los seis modelos coinciden en el 76.6 % de las imágenes de prueba; cuando coinciden aciertan el 98.3 % de las veces, y cuando discrepan cada modelo acierta en promedio solo el 63.5 %. El desacuerdo es, por tanto, un buen indicador de una imagen dudosa, y la aplicación lo muestra como «consenso».

5.6 Resultados del detector

Tabla 12. *Resultados del detector YOLOv8n en prueba.*

Métrica	Valor
mAP@50	0.979
mAP@50-95	0.723
IoU medio de la caja de mayor confianza	0.831
Imágenes con IoU ≥ 0.75 (umbral del reto)	83.3 % (de 1,553)
Tiempo de entrenamiento	20.0 min (6 épocas)
Inferencia	32.5 ms por imagen

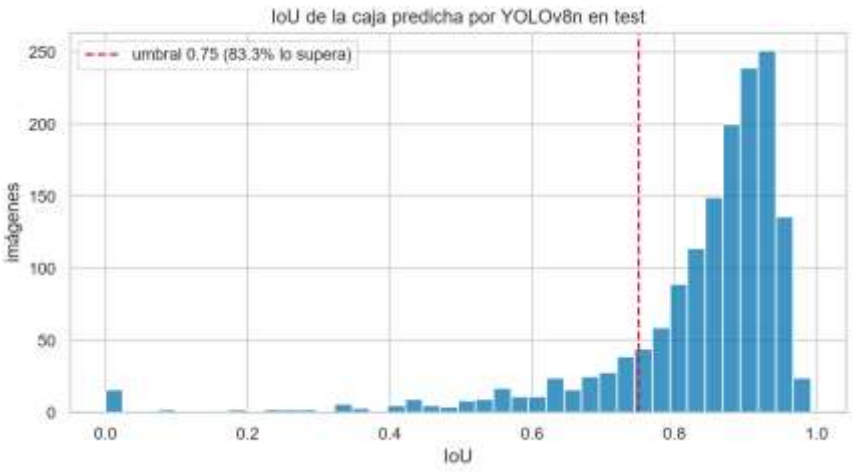


Figura 15. *Distribución del IoU de la caja predicha por YOLOv8n en prueba.*

El detector localiza el mosquito con alta fiabilidad (mAP@50 de 0.979), y en el 83.3 % de las imágenes la caja de mayor confianza supera el umbral de IoU de 0.75 del reto. Como el detector es de una sola clase, su tarea es más sencilla que la clasificación y se beneficia de que casi todas las imágenes contienen un único mosquito prominente. Un entrenamiento más largo probablemente mejoraría el mAP@50-95, que seguía subiendo en la última época. La distribución del IoU (Figura 15) se concentra entre 0.8 y 0.95, con una cola hacia valores bajos y un pequeño grupo de imágenes con IoU cercano a 0, que corresponde a fallos de localización (cajas que no se solapan con la real); el umbral de 0.75 del reto deja fuera al 16.7 % de las imágenes.

5.7 Aplicación desarrollada

Se creó una aplicación local en Streamlit (app/streamlit_app.py) en la que el backend y el frontend corren en el mismo proceso de Python, sin despliegue web. Usa los modelos de models/ y los resultados de results/, y se ejecuta con streamlit run app/streamlit_app.py (http://localhost:8501). Cuenta con las siguientes secciones:

- **Cómo funciona:** explica el flujo detector → recorte → clasificadores y los hallazgos principales.
- **Clasificar una imagen:** el usuario sube una foto (o elige una de prueba); la detección con YOLOv8n y el recorte, idéntico al del entrenamiento, ocurren de forma transparente. Puede elegir uno, varios o todos los modelos y ver el consenso entre ellos.
- **Explorar los datos:** distribución de clases y variables del conjunto limpio, y recortes por clase.
- **Rendimiento de los modelos:** gráficas interactivas con los datos de prueba, que pueden ocultarse con un interruptor en la barra lateral.

La barra lateral permite además descargar un .zip con el resumen de resultados, métricas y figuras. El script app/inference.py verifica que la aplicación reproduce las predicciones de prueba guardadas, lo que garantiza consistencia entre el entrenamiento y la aplicación.

Selección de tecnologías: se eligió Streamlit porque permite construir una interfaz interactiva únicamente con Python, reutilizando directamente el código de inferencia y sin necesidad de un servidor web

separado; Plotly aporta gráficas interactivas; y la ejecución local es coherente con que todo el entrenamiento se hizo en una sola laptop.

6. Conclusiones

A continuación se explica cómo se cumplió cada objetivo y se discute la efectividad de los algoritmos.

- **Objetivo 1 (desbalance):** el análisis mostró un desbalance de 98:1 (albopictus 4,611 imágenes frente a aegypti 47), con albopictus y Culex sumando el 88.6 % del conjunto. Esto determinó usar F1 macro como métrica, estratificar la partición antes del sobremuestreo y aplicar ponderación de clases, sobremuestreo y label smoothing.
- **Objetivo 2 (calidad y variación de las imágenes):** se documentó la variación en tamaño (139 a 9,280 px de ancho), fondo, iluminación y encuadre, y que el mosquito ocupa una mediana de solo 7.3 % de la imagen. Esto llevó al recorte con 15 % de margen y a un aumento de datos con recortes aleatorios, volteos, rotaciones y cambios de color.
- **Objetivo 3 (modelo base):** la regresión logística sobre DINOv2 (F1 macro 0.630 en prueba) y EfficientNet-B0 (0.540) sirvieron como líneas base frente a las cuales se midieron las demás variantes.
- **Objetivo 4 (comparación):** se compararon seis clasificadores con F1 macro, exactitud balanceada, exactitud, desempeño por clase, matrices de confusión y tiempos de entrenamiento e inferencia. El mejor modelo fue DINOv2 + SVM (F1 macro 0.664), seguido de cerca por la cascada jerárquica (0.662).
- **Objetivo 5 (aplicación):** se construyó una aplicación en Streamlit que integra detector y clasificadores, con comparación de modelos y consenso, y se verificó que reproduce las predicciones de prueba.
- **Objetivo general (localización y clasificación):** se cumplió en lo esencial con un detector YOLOv8n de una clase (mAP@50 de 0.979, IoU medio de 0.831 y 83.3 % de las imágenes con IoU ≥ 0.75) y un clasificador con F1 macro de 0.664. No se logró reconocer de forma fiable las especies más raras, en particular Ae. aegypti.

6.1 Discusión sobre la efectividad de los algoritmos

- Las características congeladas de DINOv2 con un clasificador simple superaron a las CNN con ajuste fino, con mucho menor costo de entrenamiento. Esto sugiere que, con pocos datos y hardware limitado, reutilizar representaciones de un modelo grande preentrenado es más eficiente que ajustar una red completa; sin embargo, las CNN recibieron un presupuesto limitado de épocas y resolución, por lo que la conclusión es válida para este contexto de cómputo.
- La exactitud (0.88–0.94) sobrestima la utilidad real: el F1 macro (0.54–0.67) muestra que el problema no está resuelto para las clases raras. Ae. aegypti, vector principal de dengue, no se reconoce en cinco de los seis modelos, lo que es una limitación importante para un uso real en salud pública.

- Los errores se concentran entre especies morfológicamente cercanas (aegypti, albopictus y japonicus/koreicus) y hacia las clases mayoritarias. Las técnicas de balanceo mejoran el recall de las clases raras, pero a costa de más falsos positivos: existe un compromiso entre ambos que depende de la aplicación (en vigilancia sanitaria puede preferirse un recall alto con revisión humana posterior).
- La estrategia de dos etapas funciona: el detector localiza bien y el clasificador trabaja sobre un recorte limpio. La combinación promediada de modelos mejora levemente (0.672), y el desacuerdo entre modelos sirve como indicador de incertidumbre.
- La hipótesis de la propagación de errores en la cascada no se confirmó: fue la única variante que recuperó algo de aegypti y mejoró Anopheles frente al modelo plano con el mismo backbone, aunque con muy pocas imágenes de prueba la evidencia es limitada.
- Con 7 imágenes de prueba de aegypti y 13 de Anopheles, las métricas de estas clases tienen alta varianza; los resultados deben interpretarse con cautela.

6.2 Limitaciones y trabajo futuro

- Evaluar el sistema de extremo a extremo: las métricas de clasificación se calcularon sobre recortes centrados en la caja anotada, por lo que el desempeño real con las cajas del detector (83.3 % con IoU ≥ 0.75) puede ser menor; la revisión bibliográfica planteaba evaluar con los recortes del detector.
- Conseguir más imágenes de las clases raras, o usar validación cruzada estratificada para estimar mejor su desempeño.
- Entrenar el detector y las CNN durante más épocas y evaluar ajuste fino de DINOv2.
- Ajustar umbrales de decisión por clase para priorizar el recall de especies vectoras.
- Probar pérdidas sensibles al desbalance (por ejemplo, focal loss) y más variantes jerárquicas.

Referencias

Buda, M., Maki, A., & Mazurowski, M. A. (2018). A systematic study of the class imbalance problem in convolutional neural networks. Neural Networks, 106, 249–259. https://doi.org/10.1016/j.neunet.2018.07.011

Cortes, C., & Vapnik, V. (1995). Support-vector networks. Machine Learning, 20, 273–297. https://doi.org/10.1007/BF00994018

Das, B. P. (2013). Pictorial key to common Culex (Culex) mosquito species associated with Japanese encephalitis virus in India. En Japanese encephalitis virus vector mosquitoes of North India (pp. 25–42). Springer. https://doi.org/10.1007/978-81-322-0861-7_34

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., … Houlsby, N. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. International Conference on Learning Representations (ICLR).

Duan, J., Liu, L., Deming, M., & Mingkong. (2026). LW-YOLO: A lightweight, enhanced YOLOv8-based model for mosquito detection. Algorithms, 19(4), Article 267. https://doi.org/10.3390/a19040267

Goodwin, A., Padmanabhan, S., Hira, S., Glancey, M., Seneviratne, S., Lemoine, S., Sahu, S. K., Bahl, E., Sahu, A., & Sahu, B. (2021). Mosquito species identification using convolutional neural networks with a multitiered ensemble model for novel species detection. Scientific Reports, 11(1), Article 14546. https://doi.org/10.1038/s41598-021-92891-9

Jocher, G., Chaurasia, A., & Qiu, J. (2023). Ultralytics YOLOv8 [Software]. https://github.com/ultralytics/ultralytics

Kittichai, V., Pengsakul, T., Chumchuen, K., Samung, Y., Sriwichai, P., Phatthamolrat, N., Tongloy, T., Jaksukam, K., Chuwongin, S., & Boonsang, S. (2021). Deep learning approaches for challenging species and gender identification of mosquito vectors. Scientific Reports, 11, Article 4838. https://doi.org/10.1038/s41598-021-84219-4

Liu, Z., Mao, H., Wu, C.-Y., Feichtenhofer, C., Darrell, T., & Xie, S. (2022). A ConvNet for the 2020s. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR).

Mosquito Alert. (s. f.). Aedes japonicus, Mosquito del Japón (nombre no oficial). Recuperado el 5 de septiembre de 2026, de https://www.mosquitoalert.com/sobre-mosquitos/aedes-japonicus/

Mosquito Alert Team, & AIcrowd. (2023). MosquitoAlert Challenge 2023 [Descripción del reto y conjunto de datos]. AIcrowd. https://www.aicrowd.com/challenges/mosquitoalert-challenge-2023

Motta, D., Santos, A. Á. B., Winkler, I., Machado, B. A. S., Pereira, D. A. D. I., Cavalcanti, A. M., Fonseca, E. O. L., Kirchner, F., & Badaró, R. (2019). Application of convolutional neural networks for classification of adult mosquitoes in the field. PLOS ONE, 14(1), Article e0210829. https://doi.org/10.1371/journal.pone.0210829

Motta, D., Santos, A. Á. B., Machado, B. A. S., Ribeiro-Filho, O. G. V., Camargo, L. O. A., Valdenegro-Toro, M. A., Kirchner, F., & Badaró, R. (2020). Optimization of convolutional neural network hyperparameters for automatic classification of adult mosquitoes. PLOS ONE, 15(7), Article e0234959. https://doi.org/10.1371/journal.pone.0234959

Nebbak, A., Almeras, L., Parola, P., & Bitam, I. (2022). Mosquito vectors (Diptera: Culicidae) and mosquito-borne diseases in North Africa. Insects, 13(10), Article 962. https://doi.org/10.3390/insects13100962

Ong, S.-Q., Ahmad, H., Nair, G., Isawasan, P., & Majid, A. H. A. (2021). Implementation of a deep learning model for automated classification of Aedes aegypti (Linnaeus) and Aedes albopictus (Skuse) in real time. Scientific Reports, 11, Article 9908. https://doi.org/10.1038/s41598-021-89365-3

Oquab, M., Darcet, T., Moutakanni, T., Vo, H., Szafraniec, M., Khalidov, V., … Bojanowski, P. (2024). DINOv2: Learning robust visual features without supervision. Transactions on Machine Learning Research. https://doi.org/10.48550/arXiv.2304.07193

Organización Mundial de la Salud. (26 de septiembre de 2024). Enfermedades transmitidas por vectores. https://www.who.int/news-room/fact-sheets/detail/vector-borne-diseases

Paixão, E. S., Teixeira, M. G., & Rodrigues, L. C. (2018). Zika, chikungunya and dengue: the causes and threats of new and re-emerging arboviral diseases. BMJ Global Health, 3(Suppl 1), e000530. https://doi.org/10.1136/bmjgh-2017-000530

Park, J., Kim, D. I., Choi, B., Kang, W., & Kwon, H. W. (2020). Classification and morphological analysis of vector mosquitoes using deep convolutional neural networks. Scientific Reports, 10, Article 1012. https://doi.org/10.1038/s41598-020-57875-1

Pataki, B. A., Garriga, J., Eritja, R., Palmer, J. R. B., Bartumeus, F., & Csabai, I. (2021). Deep learning identification for citizen science surveillance of tiger mosquitoes. Scientific Reports, 11, Article 4718. https://doi.org/10.1038/s41598-021-83657-4

Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).

Richter-Boix, A. (14 de julio de 2021). Aedes albopictus, Mosquito tigre. Mosquito Alert. https://www.mosquitoalert.com/sobre-mosquitos/mosquito-tigre/

Silla, C. N., Jr., & Freitas, A. A. (2011). A survey of hierarchical classification across different application domains. Data Mining and Knowledge Discovery, 22(1–2), 31–72. https://doi.org/10.1007/s10618-010-0175-9

Tan, M., & Le, Q. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. Proceedings of the 36th International Conference on Machine Learning (ICML).

Terven, J., Córdova-Esparza, D.-M., & Romero-González, J.-A. (2023). A comprehensive review of YOLO architectures in computer vision: From YOLOv1 to YOLOv8 and YOLO-NAS. Machine Learning and Knowledge Extraction, 5(4), 1680–1716. https://doi.org/10.3390/make5040083